Разработать, запустить и расширить функциональностью Docker-контейнерное приложение согласно варианту задания, используя предоставленную виртуальную машину с указанными в конфигурации ресурсами. Конфигурация: https://disk.yandex.ru/d/gagWU_zn1erR8g

Вариант	Задание 1 (Создание docker-compose.yml)	Задание 2 (Запуск и проверка работоспособности)	Задание 3 (Индивидуальное задание)
7	Создать файл docker-compose.yml для системы анализа посещаемости (Python Flask + InfluxDB).	Запустить приложение и проверить запись данных.	Добавить прогноз посещаемости на следующий месяц.

Решение

```
dev@dev-vm:~/task_1$ tree
.
├── app
│   ├── app.py
│   ├── forecast.py
│   ├── requirements.txt
│   └── templates
│       └── index.html
├── data
│   ├── generate_data.py
│   └── visits.csv
├── docker
│   └── Dockerfile
├── docker-compose.yml
├── docs
└── output
    └── forecast.png

6 directories, 9 files
dev@dev-vm:~/task_1$
```

Переход в проект:

cd ~/task_1

Запуск:

docker compose up –build

Что происходит:

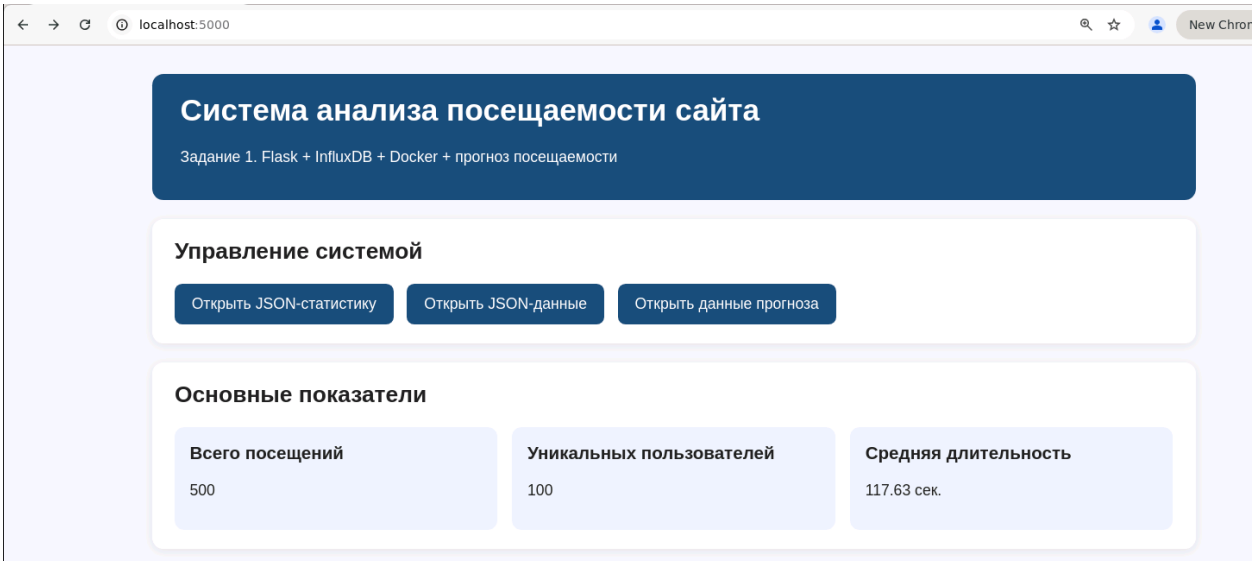
собирается контейнер с Flask (app)

запускается база данных InfluxDB

поднимается вся система

В браузере:

http://localhost:5000/



Разработать, запустить и расширить функциональностью Docker-контейнерное приложение согласно варианту задания, используя предоставленную виртуальную машину с указанными в конфигурации ресурсами. Конфигурация: https://disk.yandex.ru/d/gagWU_zn1erR8g

Вариант	Задание 1 (Создание docker-compose.yml)	Задание 2 (Запуск и проверка работоспособности)	Задание 3 (Индивидуальное задание)
6	Создать файл docker-compose.yml для системы учета	Запустить приложение и проверить обработку заказов.	Реализовать расчет среднего времени доставки.
	заказов (Django + PostgreSQL).		

Шпаргалка по выполнению задания

Создание рабочей папки проекта
mkdir -p ~/task_2
cd ~/task_2

Пояснение:
Создается основная папка задания `task_2`. Вся дальнейшая работа ведется только внутри нее.

2. Создание папки для Django-проекта
   mkdir app
   cd app

Пояснение:
В папке `app` будет лежать само веб-приложение Django.

3. Установка Python-инструментов и Django
   sudo apt update
   sudo apt install python3-pip -y
   pip3 install django

Пояснение:
Устанавливается `pip` и сам фреймворк Django.

4. Создание Django-проекта
   django-admin startproject config .

Пояснение:
Создается Django-проект с именем `config` в текущей папке.

5. Создание приложения orders
   python3 manage.py startapp orders

Пояснение:
Создается приложение `orders`, в котором будет логика заказов.

6. Первый запуск Django для проверки
   python3 manage.py runserver

Пояснение:
Проверка, что Django-проект создан правильно и запускается локально.

7. Остановка сервера
   Ctrl + C

Пояснение:
Останавливает локальный сервер Django.

8. Применение стандартных миграций Django
   python3 manage.py migrate

Пояснение:
Создаются служебные таблицы Django.

9. Создание файла зависимостей проекта
   nano requirements.txt

Пояснение:
Создается файл `requirements.txt`, где перечисляются Python-библиотеки проекта.

10. Создание Dockerfile
    nano Dockerfile

Пояснение:
Создается файл `Dockerfile`, который описывает, как собирать контейнер с Django.

11. Переход в корень проекта
    cd ~/task_2

Пояснение:
Возврат в главную папку задания, где будет лежать `docker-compose.yml`.

12. Создание docker-compose.yml
    nano docker-compose.yml

Пояснение:
Создается главный файл Docker Compose для запуска двух сервисов: Django и PostgreSQL.

13. Редактирование настроек Django
    nano ~/task_2/app/config/settings.py

Пояснение:
Настраивается подключение Django к PostgreSQL вместо SQLite.

14. Полная остановка контейнеров
    docker compose down

Пояснение:
Останавливает и удаляет контейнеры проекта перед новым запуском.

15. Сборка и запуск контейнеров
    docker compose up --build

Пояснение:
Собирает образы и запускает контейнеры приложения и базы данных.

16. Просмотр списка работающих контейнеров
    docker compose ps

Пояснение:
Показывает, какие сервисы запущены и в каком они состоянии.

17. Просмотр логов web-контейнера
    docker compose logs web

Пояснение:
Используется для поиска ошибок Django-контейнера.

18. Просмотр логов db-контейнера
    docker compose logs db

Пояснение:
Используется для проверки запуска PostgreSQL.

19. Создание миграций для своей модели
    docker compose exec web python manage.py makemigrations

Пояснение:
Django фиксирует изменения в моделях и создает файл миграции.

20. Применение миграций в PostgreSQL
    docker compose exec web python manage.py migrate

Пояснение:
Создаются таблицы в базе данных PostgreSQL.

21. Создание суперпользователя
    docker compose exec web python manage.py createsuperuser

Пояснение:
Создается администратор для входа в Django admin.

22. Редактирование модели заказов
    nano ~/task_2/app/orders/models.py

Пояснение:
Добавляется модель `Order` с полями времени создания и доставки.

23. Регистрация модели в админке
    nano ~/task_2/app/orders/admin.py

Пояснение:
Модель `Order` добавляется в панель администратора Django.

24. Создание представления для среднего времени доставки
    nano ~/task_2/app/orders/views.py

Пояснение:
Добавляется логика расчета среднего времени доставки заказов.

25. Подключение URL-адреса
    nano ~/task_2/app/config/urls.py

Пояснение:
Добавляется маршрут страницы, на которой будет показан результат расчета.

26. Запуск Django shell внутри контейнера
    docker compose exec web python manage.py shell

Пояснение:
Открывается интерактивная консоль Django для создания тестовых данных.

27. Выход из Django shell
    exit()

Пояснение:
Закрывает интерактивную консоль.

28. Проверка структуры проекта
    find . -maxdepth 2 -type f

Пояснение:
Показывает основные файлы проекта и помогает убедиться, что все создано в нужных местах.

29. Проверка содержимого файла зависимостей
    cat ~/task_2/app/requirements.txt

Пояснение:
Показывает список библиотек проекта.

30. Проверка содержимого Dockerfile
    cat ~/task_2/app/Dockerfile

Пояснение:
Показывает команды сборки контейнера Django.

31. Проверка содержимого docker-compose.yml
    cat ~/task_2/docker-compose.yml

Пояснение:
Показывает конфигурацию сервисов `web` и `db`.

32. Проверка локального адреса приложения
    http://localhost:8000

Пояснение:
Главная страница Django после запуска контейнера.

33. Проверка панели администратора
    http://localhost:8000/admin

Пояснение:
Страница входа в админку Django.

34. Проверка результата задания
    http://localhost:8000/average-delivery/

Пояснение:
Страница, на которой выводится среднее время доставки заказов.

Краткая логика выполнения задания

1. Создать папку `task_2`.
2. Внутри нее создать папку `app`.
3. Установить Django.
4. Создать Django-проект и приложение `orders`.
5. Подготовить `requirements.txt` и `Dockerfile`.
6. Создать `docker-compose.yml`.
7. Настроить PostgreSQL в `settings.py`.
8. Собрать и запустить контейнеры.
9. Создать модель заказов.
10. Выполнить миграции.
11. Создать суперпользователя.
12. Добавить расчет среднего времени доставки.
13. Создать тестовые заказы.
14. Проверить страницу с результатом.

Основные команды в правильном порядке

```
mkdir -p ~/task_2
cd ~/task_2
mkdir app
cd app
sudo apt update
sudo apt install python3-pip -y
pip3 install django
django-admin startproject config .
python3 manage.py startapp orders
python3 manage.py runserver
python3 manage.py migrate
cd ~/task_2
```

nano docker-compose.yml
nano ~/task_2/app/config/settings.py
nano ~/task_2/app/requirements.txt
nano ~/task_2/app/Dockerfile
docker compose down
docker compose up --build
docker compose ps
docker compose logs web
docker compose logs db
docker compose exec web python manage.py makemigrations
docker compose exec web python manage.py migrate
docker compose exec web python manage.py createsuperuser
nano ~/task_2/app/orders/models.py
nano ~/task_2/app/orders/admin.py
nano ~/task_2/app/orders/views.py
nano ~/task_2/app/config/urls.py
docker compose exec web python manage.py shell
find . -maxdepth 2 -type f

.

План запуска проекта после повторного включения виртуальной машины

1. Открыть терминал

Пояснение:
Запуск работы начинается с терминала Ubuntu.

2. Перейти в папку проекта

```
cd ~/task_2
```

Пояснение:
Переход в корневую директорию проекта, где находится `docker-compose.yml`.

3. Проверить, что Docker работает

```
docker --version
docker compose version
```

Пояснение:
Убедиться, что Docker и Docker Compose доступны.

4. Запустить контейнеры

```
docker compose up
```

Пояснение:
Запускает уже собранные контейнеры Django и PostgreSQL.

Если нужно пересобрать (например, были изменения):

```
docker compose up --build
```

---

5. Проверить статус контейнеров (в новом терминале)

```
docker compose ps
```

Пояснение:
Проверка, что оба контейнера (`web` и `db`) запущены.

---

6. Проверить логи при необходимости

```
docker compose logs web
docker compose logs db
```

Пояснение:
Используется, если приложение не запускается или есть ошибки.

---

7. Открыть приложение в браузере

Главная страница:
http://localhost:8000

Админка:
http://localhost:8000/admin

Страница задания:
http://localhost:8000/average-delivery/

Пояснение:
Проверка, что приложение работает и доступно.

---

8. (При необходимости) выполнить миграции

```
docker compose exec web python manage.py migrate
```

Пояснение:
Если база была сброшена или появились изменения моделей.

---

9. (При необходимости) создать суперпользователя

```
docker compose exec web python manage.py createsuperuser
```

Пояснение:
Если пользователь не сохранился или база новая.

---

10. (При необходимости) открыть Django shell

```
docker compose exec web python manage.py shell
```

Пояснение:
Для проверки данных или добавления новых заказов.

Выход:

```
exit()
```

---

11. Остановка проекта

В терминале, где запущен `docker compose up`:

```
Ctrl + C
```

Или полностью остановить контейнеры:

```
docker compose down
```

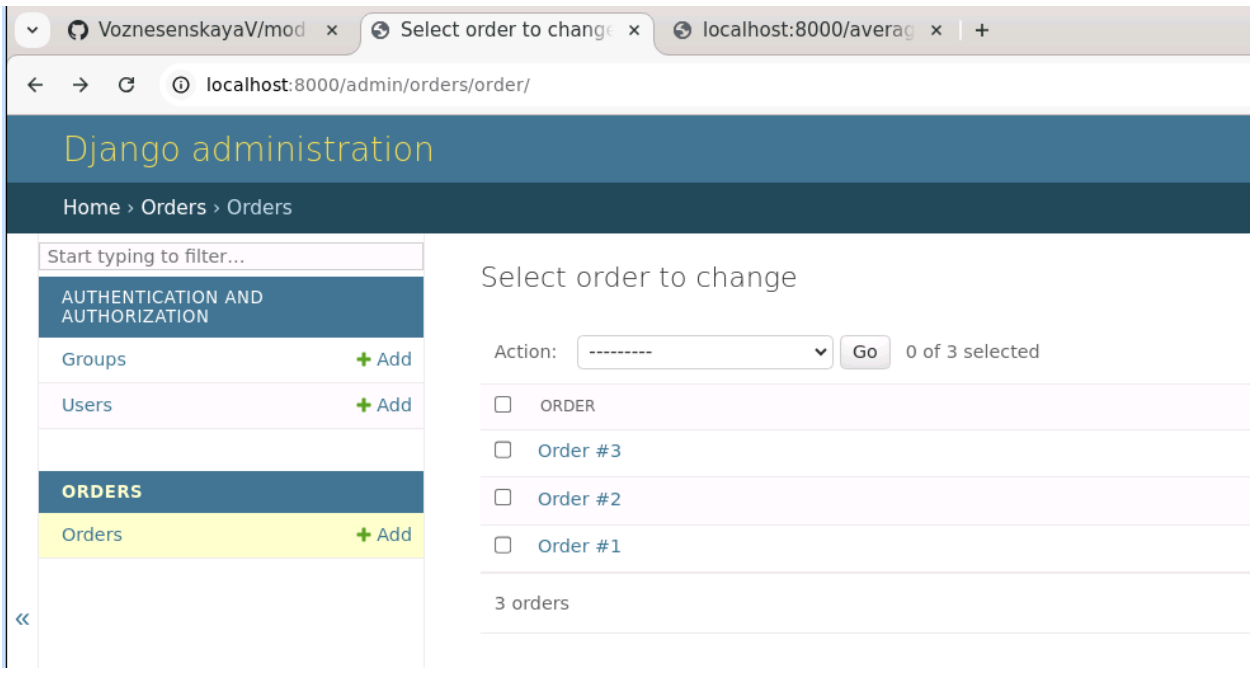
Пояснение:
Останавливает и удаляет контейнеры проекта.

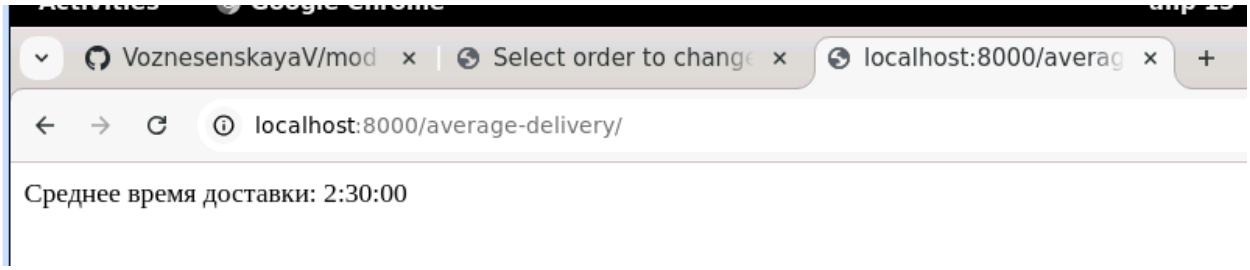
Краткая логика запуска

1. Перейти в папку `task_2`.
2. Запустить `docker compose up`.
3. Убедиться, что контейнеры работают.
4. Открыть сайт в браузере.
5. При необходимости выполнить миграции или создать пользователя.

Основные команды в правильном порядке

```
cd ~/task_2
docker compose up
docker compose ps
docker compose logs web
docker compose exec web python manage.py migrate
docker compose exec web python manage.py createsuperuser
docker compose exec web python manage.py shell
docker compose down
```

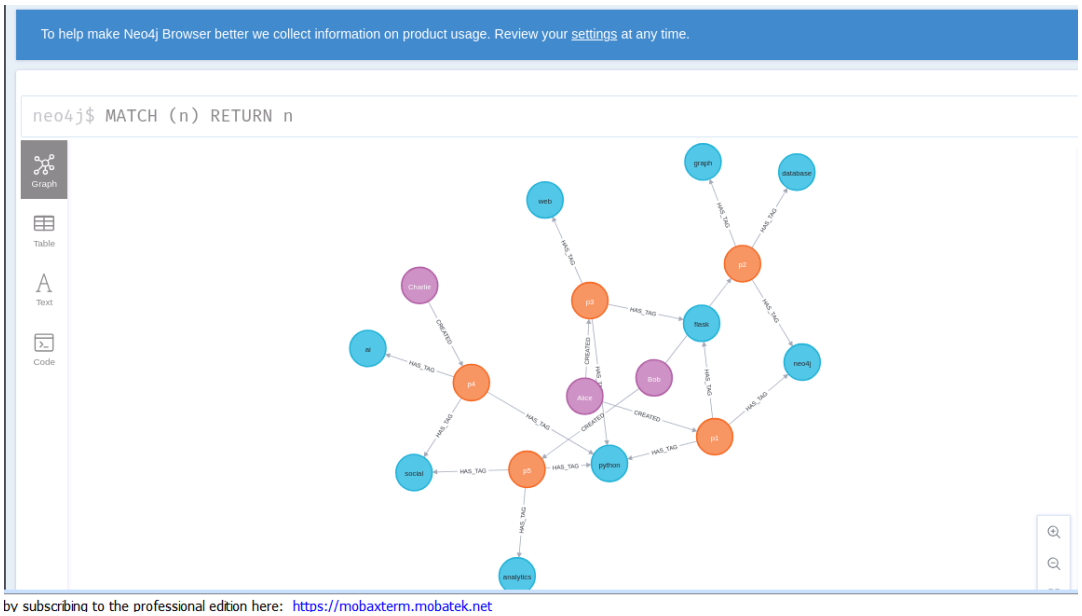

VoznesenskayaV/mod
Select order to change
localhost:8000/averag
localhost:8000/admin/orders/order/
Django administration
Home › Orders › Orders
Start typing to filter...
AUTHENTICATION AND AUTHORIZATION
Groups + Add
Users + Add
ORDERS
Orders + Add
Select order to change
Action: --------- Go 0 of 3 selected
ORDER
Order #3
Order #2
Order #1
3 orders


VoznesenskayaV/mod
Select order to change
localhost:8000/averag
localhost:8000/average-delivery/
Среднее время доставки: 2:30:00

Разработать, запустить и расширить функциональностью Docker-контейнерное приложение согласно варианту задания, используя предоставленную виртуальную машину с указанными в конфигурации ресурсами. Конфигурация: https://disk.yandex.ru/d/gagWU_zn1erR8g

Вариант	Задание 1 (Создание docker-compose.yml)	Задание 2 (Запуск и проверка работоспособности)	Задание 3 (Индивидуальное задание)
5	Создать файл docker-compose.yml для системы анализа социальных сетей (Python Flask + Neo4j).	Запустить приложение и проверить графовые запросы.	Добавить анализ популярности хештегов.

MATCH (n) RETURN n



Видим узлы:

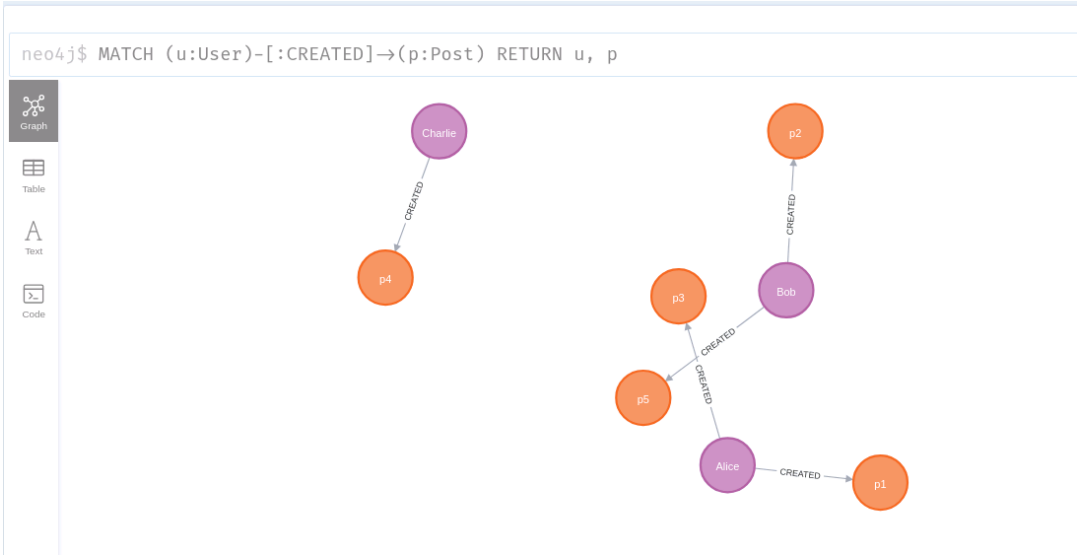
User

Post

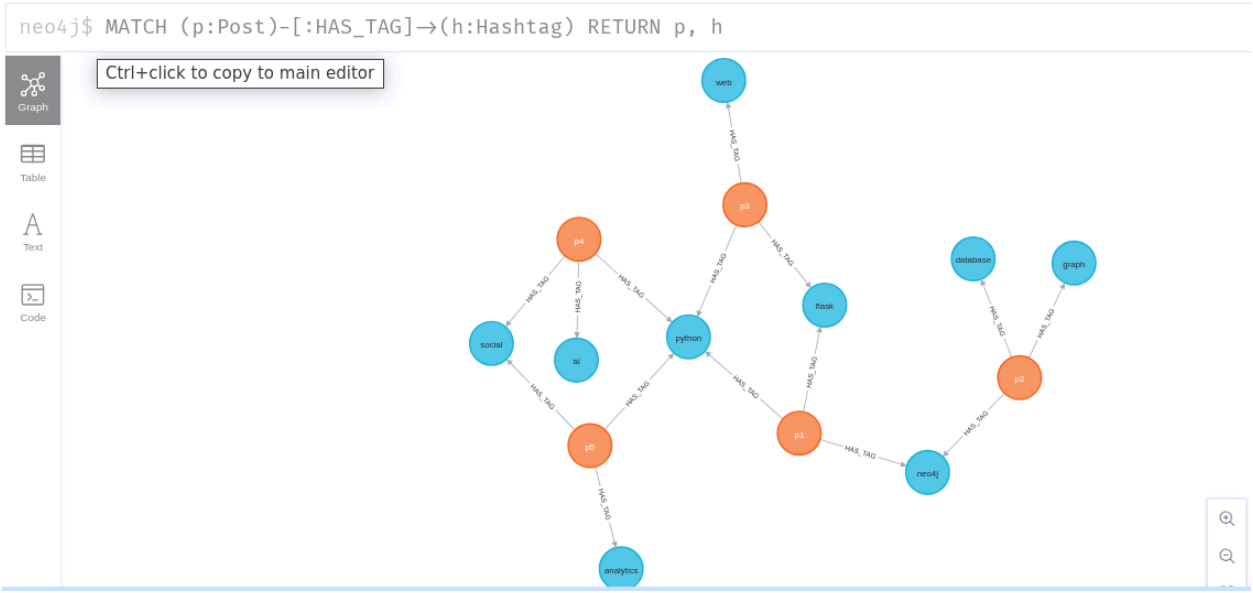
Hashtag

Это уже доказательство, что данные реально в графе.

MATCH (u:User)-[:CREATED]->(p:Post) RETURN u, p



MATCH (p:Post)-[:HAS_TAG]->(h:Hashtag) RETURN p, h



Эти показывают связи:

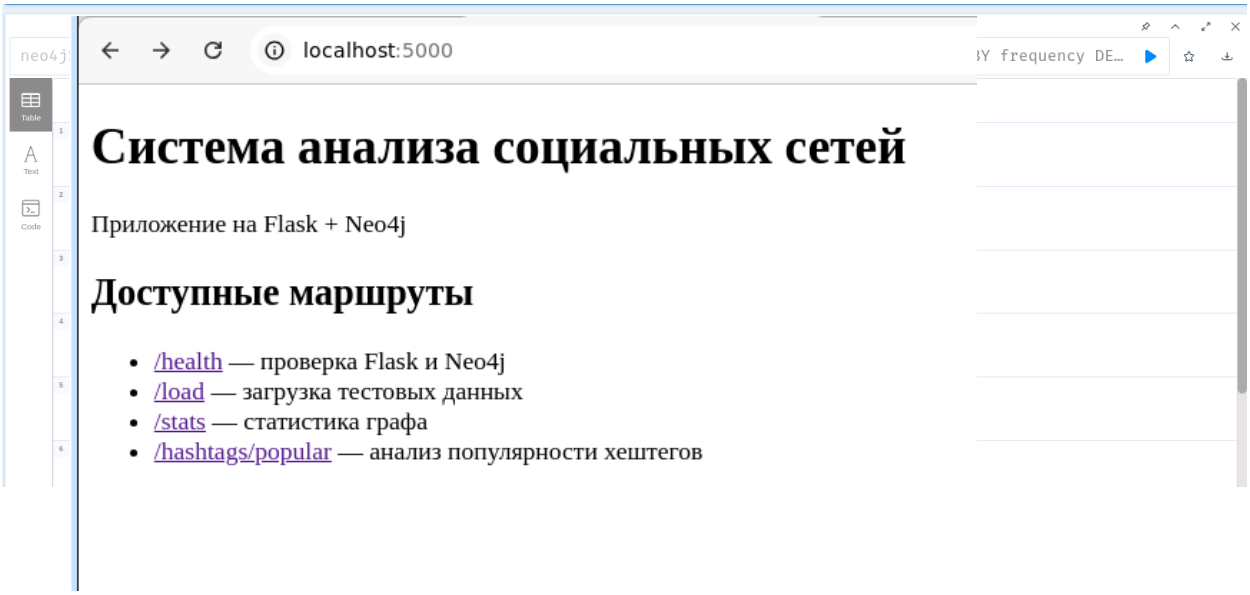
1) кто создал пост
2) какие хештеги есть у постов

Таблица популярности хэштегов:

MATCH (p:Post)-[:HAS_TAG]->(h:Hashtag)

RETURN h.name AS hashtag, count(*) AS frequency

ORDER BY frequency DESC, hashtag ASC



Шаги реализации:

cd ~ перейти в домашнюю директорию
mkdir -p task_3 создать папку проекта
cd task_3 перейти в папку проекта
mkdir -p app/templates app/static data cypher screenshots reports создать структуру папок проекта
touch app/app.py app/requirements.txt app/templates/index.html data/sample_posts.json cypher/init.cypher cypher/test_queries.cypher Dockerfile docker-compose.yml .env README.md .gitignore создать все необходимые файлы

docker --version проверить установлен ли Docker
docker compose version проверить установлен ли Docker Compose
docker run hello-world проверить что Docker работает

nano app/requirements.txt записать зависимости Python (Flask и neo4j)
nano app/app.py написать код Flask приложения
nano app/templates/index.html создать главную HTML страницу

python3 --version проверить установлен ли Python
pip3 --version проверить установлен ли pip

pip3 install -r app/requirements.txt установить зависимости

cd app перейти в папку приложения
python3 app.py проверить что Flask работает без Docker

cd ~/task_3 вернуться в корень проекта
nano Dockerfile описать сборку контейнера Flask
nano .env задать переменные подключения к Neo4j
nano docker-compose.yml описать запуск Flask и Neo4j
nano .gitignore исключить лишние файлы

docker compose up --build собрать и запустить контейнеры

docker compose ps проверить что контейнеры работают
docker compose logs web посмотреть логи Flask
docker compose logs neo4j посмотреть логи Neo4j

docker compose down остановить контейнеры

nano app/app.py добавить подключение к Neo4j и маршруты
nano app/templates/index.html обновить список маршрутов

docker compose up --build пересобрать контейнеры после изменений

docker compose logs web проверить ошибки Flask при необходимости

cd ~/task_3
find . проверить структуру проекта

nano cypher/test_queries.cypher записать тестовые графовые запросы
nano cypher/init.cypher записать инициализацию графа
nano data/sample_posts.json добавить тестовые данные

docker compose up --build финальный запуск проекта

docker compose ps убедиться что все сервисы работают

docker compose down остановить проект после проверки

Шаги по запуску:

cd ~/task_3 перейти в папку проекта
docker compose up --build запустить и пересобрать контейнеры

http://localhost:5000 открыть главную страницу Flask
http://localhost:5000/health проверить работу Flask и подключение к Neo4j
http://localhost:5000/load загрузить тестовые данные в граф
http://localhost:5000/stats посмотреть статистику по графу
http://localhost:5000/hashtags/popular посмотреть анализ популярности хештегов

http://localhost:7474 открыть Neo4j Browser и войти в базу

docker compose ps проверить что контейнеры запущены

docker compose logs web посмотреть логи Flask при необходимости
docker compose logs neo4j посмотреть логи Neo4j при необходимости

docker compose down остановить контейнеры

docker compose up запустить контейнеры без пересборки

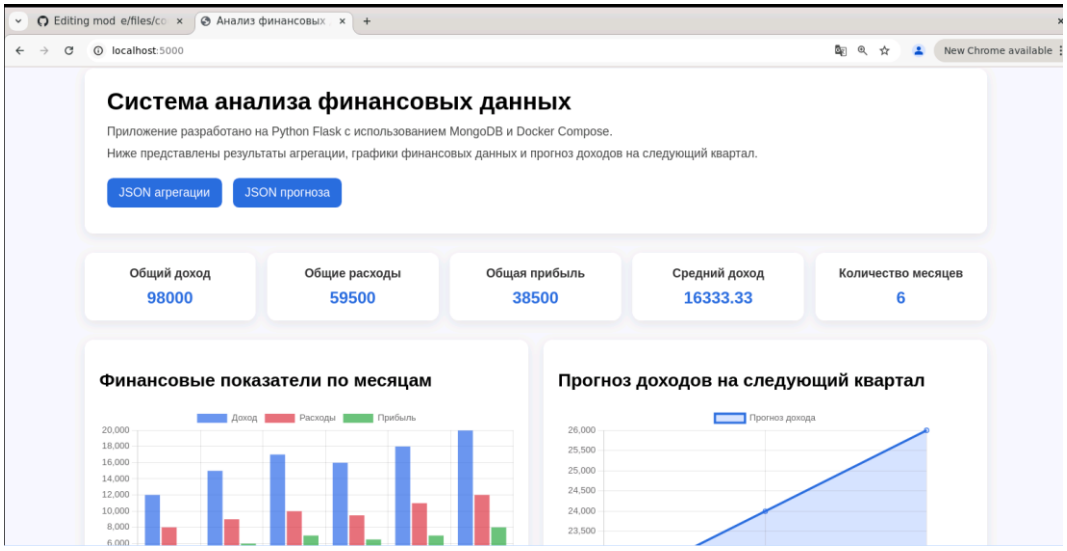
docker compose restart перезапустить контейнеры

docker compose stop остановить контейнеры без удаления

docker compose start снова запустить контейнеры

Разработать, запустить и расширить функциональностью Docker-контейнерное приложение согласно варианту задания, используя предоставленную виртуальную машину с указанными в конфигурации ресурсами. Конфигурация: https://disk.yandex.ru/d/gagWU_zn1erR8g

Вариант	Задание 1 (Создание docker-compose.yml)	Задание 2 (Запуск и проверка работоспособности)	Задание 3 (Индивидуальное задание)
4	Создать файл docker-compose.yml для системы анализа финансовых данных (Python Flask + MongoDB).	Запустить приложение и проверить агрегацию данных.	Добавить прогноз доходов на следующий квартал.




```
dev@dev-vm:~/task_4$ tree
.
├── app
│   ├── app.py
│   ├── Dockerfile
│   ├── requirements.txt
│   └── templates
│       └── index.html
├── data
│   └── init_data.json
├── docker-compose.yml
└── mongo
    └── init.js

4 directories, 7 files
dev@dev-vm:~/task_4$
```

Шпаргалка по выполнению задания

Все делать внутри папки task_4.

1. Создать структуру проекта
   Это нужно, чтобы сразу подготовить все папки и файлы проекта.

Команда:
mkdir -p task_4/{app/templates,data,mongo} && touch task_4/docker-compose.yml task_4/app/{app.py,requirements.txt,Dockerfile} task_4/mongo/init.js task_4/app/templates/index.html

Перейти в папку проекта:
cd task_4

Проверить структуру:
tree

Если tree не установлен:
sudo apt update
sudo apt install tree -y

2. Заполнить файлы проекта
   Нужно последовательно открыть и заполнить файлы:
   app/requirements.txt
   mongo/init.js
   app/Dockerfile
   docker-compose.yml
   app/app.py
   app/templates/index.html

Для редактирования удобно использовать:
nano app/requirements.txt
nano mongo/init.js
nano app/Dockerfile
nano docker-compose.yml
nano app/app.py
nano app/templates/index.html

Кратко по назначению файлов

app/requirements.txt
Список Python-зависимостей, которые будут установлены в контейнер Flask.

mongo/init.js
Скрипт начальной загрузки данных в MongoDB при первом запуске контейнера.

app/Dockerfile
Инструкция по сборке контейнера Flask-приложения.

docker-compose.yml
Файл для запуска двух сервисов: Flask и MongoDB.

app/app.py
Основная логика приложения: подключение к базе, агрегация, прогноз, маршруты.

app/templates/index.html
HTML-страница для отображения данных, агрегации и графиков в браузере.

3. Проверить наличие Docker
   Нужно убедиться, что Docker и Docker Compose доступны.

Команды:
docker --version
docker compose version

4. Собрать и запустить проект
   Это запускает контейнер MongoDB и контейнер Flask.

Команда:
docker compose up --build

Если запуск прошел успешно, приложение будет доступно на порту 5000.

5. Проверить, что контейнеры работают
   Это нужно, чтобы убедиться, что оба сервиса поднялись.

Команда:
docker ps

6. Проверить приложение в браузере
   Нужно открыть главную страницу и убедиться, что интерфейс отображается.

Адрес:
http://localhost:5000

На главной странице должны быть:
таблица финансовых данных
карточки с итоговой агрегацией
график финансовых показателей по месяцам
график прогноза доходов на следующий квартал

7. Проверить приложение через curl
   Это нужно для подтверждения, что маршруты работают и возвращают данные.

Проверка главной страницы:
curl http://localhost:5000/

Проверка агрегации:
curl http://localhost:5000/aggregate

Проверка прогноза:
curl http://localhost:5000/forecast

8. Что означает проверка агрегации
   Маршрут /aggregate должен показывать:
   общий доход
   общие расходы
   общую прибыль
   средний доход
   количество месяцев

Это подтверждает выполнение пункта о запуске и проверке агрегации данных.

9. Что означает проверка прогноза
   Маршрут /forecast должен показывать прогноз на 3 следующих месяца и общий итог на квартал.

В итоговой версии прогноз лучше делать по тренду роста, чтобы линия на графике была возрастающей, а не ровной.

10. Если внесены изменения в app.py или HTML
    После любых изменений приложение нужно пересобрать и запустить снова.

Команды:
docker compose down
docker compose up --build

11. Если нужно полностью пересоздать проект
    Это полезно, если база уже была создана и init.js больше не выполняется.

Команды:
docker compose down -v
docker compose up --build

12. Если нужно посмотреть логи
    Это помогает найти ошибки запуска Flask или MongoDB.

Общие логи:
docker compose logs

Логи Flask:
docker compose logs web

Логи MongoDB:
docker compose logs mongo

Что говорить по этапам выполнения
Сначала была создана структура проекта в папке task_4.
Потом подготовлены файлы для Flask, MongoDB, Dockerfile и Docker Compose.
Далее был настроен init.js для автоматической загрузки тестовых финансовых данных в MongoDB.
После этого проект был собран и запущен через docker compose up --build.
Затем была проверена работа приложения в браузере и через curl.
Маршрут /aggregate подтвердил агрегацию данных.

Маршрут /forecast подтвердил расчет прогноза доходов на следующий квартал.
На главной странице данные были визуализированы в виде таблицы, карточек и графиков.

13. Главное по пунктам задания
    Пункт 1 выполнен через создание docker-compose.yml для Flask и MongoDB.
    Пункт 2 выполнен через запуск контейнеров и проверку маршрута /aggregate.
    Пункт 3 выполнен через добавление маршрута /forecast и отображение прогноза на графике.
14. Полезные команды для быстрого пользования

Перейти в проект:
cd ~/task_4

Запустить:
docker compose up --build

Остановить:
docker compose down

Остановить и удалить тома:
docker compose down -v

Проверить контейнеры:
docker ps

Посмотреть логи:
docker compose logs

Проверить агрегацию:
curl http://localhost:5000/aggregate

Проверить прогноз:
curl http://localhost:5000/forecast

Открыть в браузере:
http://localhost:5000

Чтобы запустить уже готовый проект после включения виртуальной машины:

Перейти в папку проекта
cd ~/task_4

Проверить, что Docker запущен
docker ps

Если ошибка или пусто, запустить Docker
sudo systemctl start docker

Запустить приложение
docker compose up

Если хочешь запустить в фоне (без логов)
docker compose up -d

Проверить, что контейнеры работают
docker ps

Открыть приложение в браузере
http://localhost:5000

Проверить через терминал
curl http://localhost:5000/aggregate
curl http://localhost:5000/forecast

Если нужно остановить
docker compose down

Если что-то не работает, посмотреть логи
docker compose logs

Если были изменения в коде и нужно пересобрать
docker compose down
docker compose up --build

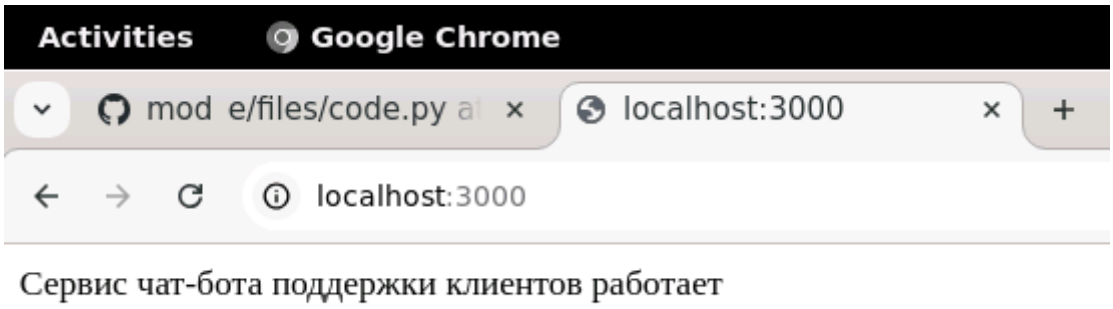
Разработать, запустить и расширить функциональностью Docker-контейнерное приложение согласно варианту задания, используя предоставленную виртуальную машину с указанными в конфигурации ресурсами. Конфигурация: https://disk.yandex.ru/d/gagWU_zn1erR8g

Вариант	Задание 1 (Создание docker-compose.yml)	Задание 2 (Запуск и проверка работоспособности)	Задание 3 (Индивидуальное задание)
3	Создать файл docker-compose.yml для чат-бота поддержки клиентов (Node.js + MySQL).	Запустить приложение и проверить логирование сообщений.	Реализовать анализ частоты обращений клиентов.

Логирование

cat logs/messages.log

```
-vm:~/task_5$ cat logs/messages.log
[2026-04-15T02:46:08.863Z] Клиент: Иван, сообщение: Не работает вход в личный кабинет
[2026-04-15T02:46:30.478Z] Клиент: Мария, сообщение: Как изменить тариф?
[2026-04-15T02:46:42.049Z] Клиент: Иван, сообщение: Когда ответит оператор?
dev@dev-vm:~/task_5$
```



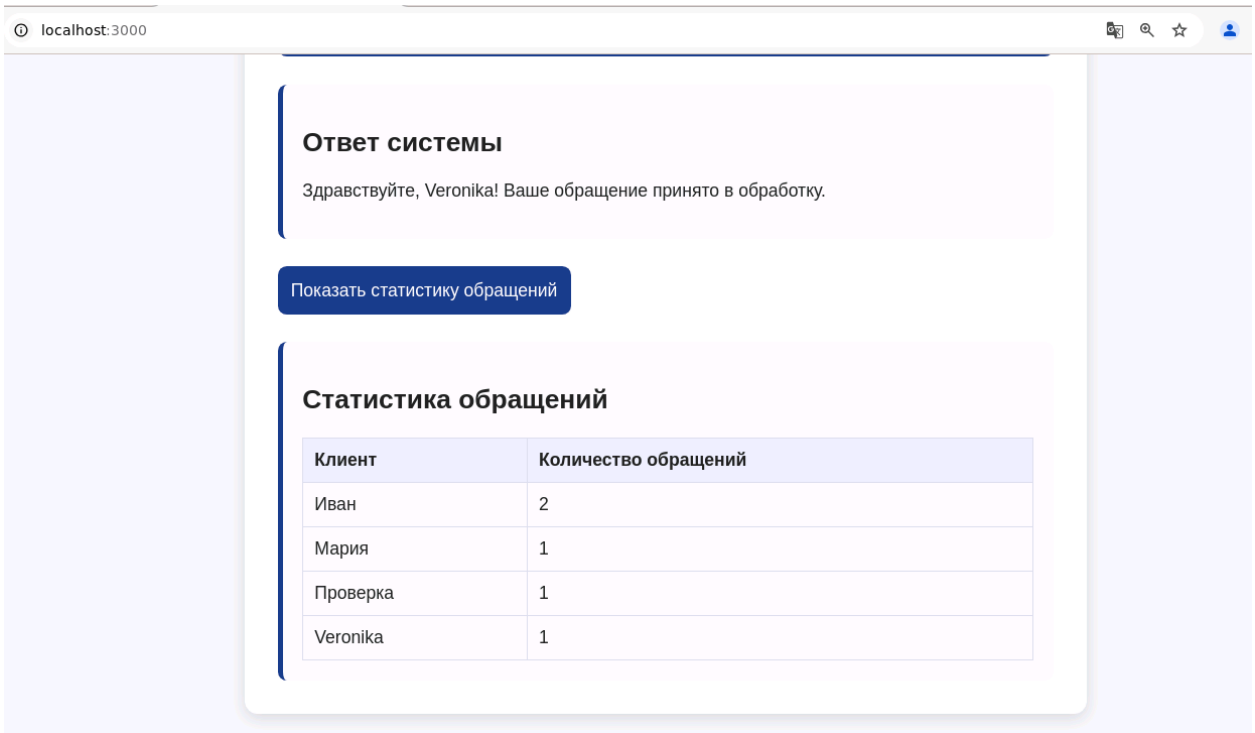
docker exec -it support_bot_db mysql -u botuser -pbotpass123 support_bot_db

SET NAMES utf8mb4;

SELECT * FROM messages;

```
mysql> SET NAMES utf8mb4;
Query OK, 0 rows affected (0.00 sec)

mysql> SELECT * FROM messages;
+----+-------------+-------------------------------------------+---------------------+
| id | client_name | message_text                              | created_at          |
+----+-------------+-------------------------------------------+---------------------+
|  1 | Иван    | Не работает вход              | 2026-04-15 02:53:49 |
|  2 | Мария  | Как изменить тариф?         | 2026-04-15 02:55:47 |
|  3 | Иван    | Когда ответит оператор? | 2026-04-15 02:55:57 |
+----+-------------+-------------------------------------------+---------------------+
3 rows in set (0.00 sec)
```



```
dev@dev-vm:~/task_5$ tree
.
├── app
│   ├── config
│   │   └── db.js
│   ├── Dockerfile
│   ├── package.json
│   ├── public
│   │   ├── index.html
│   │   ├── script.js
│   │   └── style.css
│   ├── routes
│   │   └── messages.js
│   └── server.js
├── docker-compose.yml
├── docs
├── logs
│   └── messages.log
└── mysql
    └── init
        └── init.sql

8 directories, 11 files
dev@dev-vm:~/task_5$
```

Запуск:

cd ~/task_5
перейти в папку проекта

docker compose up -d
запустить приложение и базу

docker compose ps
проверить, что контейнеры работают

docker compose logs app
посмотреть логи приложения

docker compose logs -f app
смотреть логи в реальном времени

docker compose down
остановить контейнеры

docker compose up --build -d
пересобрать и заново запустить (если меняли код)

docker compose down -v
полностью удалить контейнеры и базу (если проблемы)

curl http://localhost:3000
проверить, отвечает ли сервер

открыть в браузере
http://localhost:3000
запустить интерфейс

curl -X POST http://localhost:3000/message -H "Content-Type: application/json" -d '{"client_name":"Тест","message_text":"Проверка"}'
проверка отправки сообщения без интерфейса

cat logs/messages.log
проверить, записываются ли обращения в лог

docker exec -it support_bot_db mysql -u botuser -pbotpass123 support_bot_db
подключиться к базе

SELECT * FROM messages;
посмотреть сохраненные обращения

exit;
выйти из базы

curl http://localhost:3000/stats
проверить статистику обращений

минимальный запуск после включения ВМ:
cd ~/task_5
docker compose up -d